Practical 6

Problem Definition

Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.

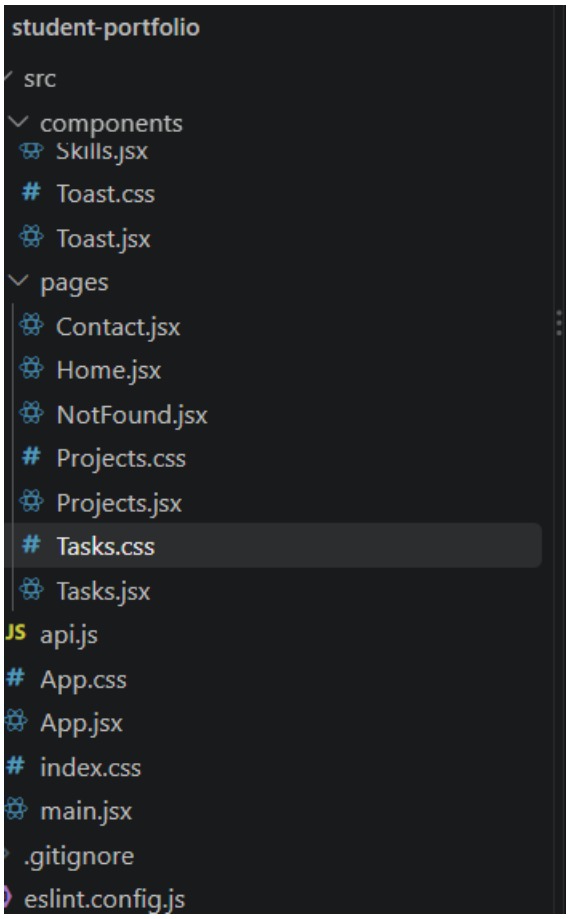
• Replace the GitHub repo data from Practical 3 with task data from your own backend.

• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.

• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.

• Handle loading and error states for every API interaction, not just the initial fetch.

Folder Structure:



backend/Practical5/server.js

```
/**
 * Practical 5 – MongoDB Integration and Schema Design with Mongoose
 *
 * Objective: Replace the in-memory array from Practical 4 with a real
 * MongoDB database accessed through the Mongoose ODM.
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

// Mongoose model
const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;

/* ─────────────────────────────────────────────────
   Middleware Pipeline
──────────────────────────────────────────────────── */

// 1. Allow React frontend to access this backend
app.use(cors());
```

```
// 2. Parse JSON request bodies
app.use(express.json());

// 3. Request logger
app.use((req, res, next) => {
  console.log(
    `[${new Date().toISOString()}] ${req.method} ${req.url}`
  );
  next();
});

// 4. Content-Type check for write operations
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT
requests'
    });
  }

  next();
});

/* ————————————————————————————————————
```

```
   Helper: Format Mongoose validation errors
──────────────────────────────────────────────────────────── */

function formatValidationError(err) {
   const errors = {};

   Object.keys(err.errors).forEach((field) => {
      errors[field] = err.errors[field].message;
   });

   return errors;
}

/* ────────────────────────────────────────────────────────────
   CRUD Routes
──────────────────────────────────────────────────────────── */

// GET /tasks – return all tasks
app.get('/tasks', async (req, res, next) => {
   try {
      const tasks = await Task.find().sort({ createdAt: -1 });

      res.status(200).json(tasks);
   } catch (err) {
      next(err);
   }
```

```
});

// GET /tasks/:id – return a single task
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
    }

    res.status(200).json(task);
  } catch (err) {

    // Invalid MongoDB ObjectId
    if (err.name === 'CastError') {
      return res.status(400).json({
        error: 'Invalid task ID format'
      });
    }

    next(err);
  }
});
```

```
// POST /tasks – create a new task
app.post('/tasks', async (req, res, next) => {
  try {
    const task = await Task.create({
      title: req.body.title,
      description: req.body.description,
      completed: req.body.completed,
      priority: req.body.priority
    });

    res.status(201).json(task);
  } catch (err) {

    // Mongoose validation error
    if (err.name === 'ValidationError') {
      return res.status(400).json({
        errors: formatValidationError(err)
      });
    }

    next(err);
  }
});

// PUT /tasks/:id – update an existing task
```

```
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      {
        title: req.body.title,
        description: req.body.description,
        completed: req.body.completed,
        priority: req.body.priority
      },
      {
        new: true,
        runValidators: true
      }
    );

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
    }

    res.status(200).json(task);
  } catch (err) {

    // Mongoose validation error
```

```
    if (err.name === 'ValidationError') {
      return res.status(400).json({
        errors: formatValidationError(err)
      });
    }

    // Invalid MongoDB ObjectId
    if (err.name === 'CastError') {
      return res.status(400).json({
        error: 'Invalid task ID format'
      });
    }

    next(err);
  }
});

// DELETE /tasks/:id – delete a task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
```

```
        }

        res.status(200).json({
            message: 'Task deleted successfully',
            task
        });
    } catch (err) {

        // Invalid MongoDB ObjectId
        if (err.name === 'CastError') {
            return res.status(400).json({
                error: 'Invalid task ID format'
            });
        }

        next(err);
    }
});

/* ————————————————————————————————————
   404 Handler – undefined routes
———————————————————————————————————— */

app.use((req, res) => {
    res.status(404).json({
        error: `Route ${req.method} ${req.url} not found`
```

```
    });
});

/* ————————————————————————————————————
   Global Error Handler
   Must be the last middleware
———————————————————————————————————— */

app.use((err, req, res, next) => {
    console.error(`[ERROR] ${err.stack}`);

    res.status(500).json({
        error: 'Something went wrong on the server'
    });
});

/* ————————————————————————————————————
   Connect to MongoDB, then start the server
———————————————————————————————————— */

mongoose
    .connect(process.env.MONGO_URI)
    .then(() => {
        console.log('MongoDB connected successfully');

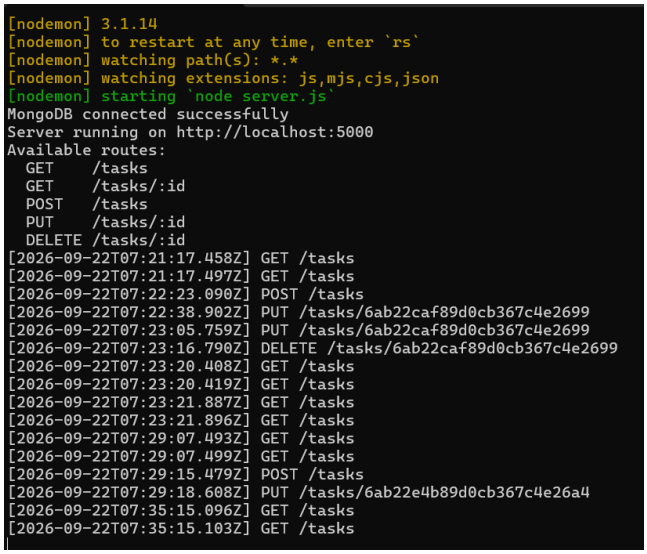
        app.listen(PORT, () => {
```

```
      console.log(`Server running on http://localhost:${PORT}`);

      console.log('Available routes:');
      console.log('  GET    /tasks');
      console.log('  GET    /tasks/:id');
      console.log('  POST   /tasks');
      console.log('  PUT    /tasks/:id');
      console.log('  DELETE /tasks/:id');
    });
  })
  .catch((err) => {
    console.error(
      'MongoDB connection failed:',
      err.message
    );

    console.error(
      'Make sure your MONGO_URI in .env is correct.'
    );

    process.exit(1);
  });
```



frontend/src/api.js

```
const BASE_URL = "http://localhost:5000";


async function handleResponse(res) {
  if (!res.ok) {
    let message = `Request failed with status ${res.status}`;
    try {
      const data = await res.json();
      message = data.error || (data.errors && Object.values(data.errors).join(",
")) || message;
    } catch {
      // response had no JSON body — keep the default message
    }
    throw new Error(message);
  }
  return res.json();
}
```

```
export const getTasks = () =>
  fetch(`${BASE_URL}/tasks`).then(handleResponse);

export const createTask = (task) =>
  fetch(`${BASE_URL}/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  }).then(handleResponse);

export const updateTask = (id, task) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  }).then(handleResponse);

export const deleteTask = (id) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "DELETE",
  }).then(handleResponse);
```

frontend/src/pages/Tasks.jsx

```
import { useState, useEffect } from "react";
import { getTasks, createTask, updateTask, deleteTask } from "../api";
import LoadingSpinner from "../components/LoadingSpinner";
import ErrorMessage from "../components/ErrorMessage";
```

```
import Toast from "../components/Toast";
import "./Tasks.css";

const emptyForm = { title: "", description: "", priority: "medium" };

function Tasks() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [loadError, setLoadError] = useState(null);

  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);
  const [submitting, setSubmitting] = useState(false);
  const [actionError, setActionError] = useState(null);
  const [deletingId, setDeletingId] = useState(null);

  const [toasts, setToasts] = useState([]);

  const addToast = (message, type = "success") => {
    const id = Date.now() + Math.random();
    setToasts((prev) => [...prev, { id, message, type }]);
  };

  const dismissToast = (id) => {
    setToasts((prev) => prev.filter((t) => t.id !== id));
  };
```

```
const loadTasks = () => {
  setLoading(true);
  setLoadError(null);
  getTasks()
    .then(setTasks)
    .catch((err) => setLoadError(err.message))
    .finally(() => setLoading(false));
};

useEffect(() => {
  loadTasks();
}, []);

const handleFormChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};

const startEdit = (task) => {
  setEditingId(task._id);
  setForm({
    title: task.title,
    description: task.description || "",
    priority: task.priority,
  });
  setActionError(null);
```

```
  };

  const cancelEdit = () => {
    setEditingId(null);
    setForm(emptyForm);
    setActionError(null);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.title.trim()) {
      setActionError("Title is required");
      return;
    }

    setActionError(null);

    // — Editing an existing task: wait for the server, same as before —
    if (editingId) {
      setSubmitting(true);
      const currentTask = tasks.find((t) => t._id === editingId);
      updateTask(editingId, { ...form, completed: currentTask?.completed ?? false
})
        .then((savedTask) => {
          setTasks((prev) => prev.map((t) => (t._id === editingId ? savedTask : t)));
          setForm(emptyForm);
          setEditingId(null);
```

```
        addToast("Task updated");
      })
      .catch((err) => {
        setActionError(err.message);
        addToast(`Update failed: ${err.message}`, "error");
      })
      .finally(() => setSubmitting(false));
    return;
  }

  // — Creating a new task: show it immediately (optimistic), reconcile after —
  const tempId = `temp-${Date.now()}`;
  const optimisticTask = {
    _id: tempId,
    title: form.title,
    description: form.description,
    priority: form.priority,
    completed: false,
    isPending: true,
  };

  setTasks((prev) => [optimisticTask, ...prev]);
  setForm(emptyForm);
  setSubmitting(true);

  createTask({ title: optimisticTask.title, description: optimisticTask.description,
priority: optimisticTask.priority })
```

```
    .then((savedTask) => {
      setTasks((prev) => prev.map((t) => (t._id === tempId ? savedTask : t)));
      addToast("Task added");
    })
    .catch((err) => {
      // Roll back the optimistic card since the server rejected it
      setTasks((prev) => prev.filter((t) => t._id !== tempId));
      setActionError(err.message);
      addToast(`Add failed: ${err.message}`, "error");
    })
    .finally(() => setSubmitting(false));
};

const toggleCompleted = (task) => {
  setActionError(null);
  updateTask(task._id, { ...task, completed: !task.completed })
    .then((updated) => {
      setTasks((prev) => prev.map((t) => (t._id === task._id ? updated : t)));
    })
    .catch((err) => {
      setActionError(err.message);
      addToast(`Update failed: ${err.message}`, "error");
    });
};

const handleDelete = (task) => {
```

```
  const confirmed = window.confirm(`Delete "${task.title}"? This can't be undone.`);
  if (!confirmed) return;

  setDeletingId(task._id);
  setActionError(null);
  deleteTask(task._id)
    .then(() => {
      setTasks((prev) => prev.filter((t) => t._id !== task._id));
      if (editingId === task._id) cancelEdit();
      addToast("Task deleted");
    })
    .catch((err) => {
      setActionError(err.message);
      addToast(`Delete failed: ${err.message}`, "error");
    })
    .finally(() => setDeletingId(null));
 };

 if (loading) return <LoadingSpinner label="Loading tasks..." />;
 if (loadError) return <ErrorMessage message={loadError} onRetry={loadTasks} />;

 return (
  <div className="page">
    <Toast toasts={toasts} onDismiss={dismissToast} />
```

```
    <div className="bento-card span-3 contact-card">
      <h2 className="card-title">{editingId ? "Update Task" : "New Task"}</h2>
      <form className="contact-form" onSubmit={handleSubmit}>
        <label>
          Title
          <input
            type="text"
            name="title"
            value={form.title}
            onChange={handleFormChange}
            placeholder="Task title"
          />
        </label>
        <label>
          Description
          <textarea
            name="description"
            value={form.description}
            onChange={handleFormChange}
            placeholder="Details (optional)"
            rows={3}
          />
        </label>
        <label>
          Priority
          <select name="priority" value={form.priority}
onChange={handleFormChange}>
```

```
            <option value="low">Low</option>
            <option value="medium">Medium</option>
            <option value="high">High</option>
          </select>
        </label>

        {actionError && <p className="task-error">{actionError}</p>}

        <div className="task-form-actions">
          <button type="submit" className="btn btn-primary"
disabled={submitting}>
            {submitting ? "Saving..." : editingId ? "Update Task" : "Add Task"}
          </button>
          {editingId && (
            <button type="button" className="btn btn-secondary"
onClick={cancelEdit}>
              Cancel
            </button>
          )}
        </div>
      </form>
    </div>

    <div className="bento-grid task-grid">
      {tasks.length === 0 && <p className="tasks-empty">No tasks yet — add
one above.</p>}
```

```
      {tasks.map((task) => (
        <div
          key={task._id}
          className={`bento-card span-3 task-card ${task.completed ? "task-completed" : ""} ${task.isPending ? "task-pending" : ""}`}
        >
          <div className="task-card-header">
            <label className="task-checkbox">
              <input
                type="checkbox"
                checked={task.completed}
                disabled={task.isPending}
                onChange={() => toggleCompleted(task)}
              />
              <h2 className="card-title">{task.title}</h2>
            </label>
            <span className={`tech-tag priority-${task.priority}`}>
              {task.isPending ? "saving..." : task.priority}
            </span>
          </div>

          {task.description && <p className="project-points">{task.description}</p>}

          {!task.isPending && (
            <div className="task-actions">
```

```
            <button className="btn btn-secondary" onClick={() =>
startEdit(task)}>
              Edit
            </button>
            <button
              className="btn btn-secondary task-delete"
              onClick={() => handleDelete(task)}
              disabled={deletingId === task._id}
            >
              {deletingId === task._id ? "Deleting..." : "Delete"}
            </button>
          </div>
        )}
      </div>
    ))}
  </div>
 </div>
 );
}

export default Tasks;
```

frontend/src/components/Toast.jsx

```
import { useEffect } from "react";
import "./Toast.css";

export default function Toast({ toasts, onDismiss }) {
 return (
```

```
    <div className="toast-stack">
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} onDismiss={onDismiss} />
      ))}
    </div>
  );
}

function ToastItem({ toast, onDismiss }) {
  useEffect(() => {
    const timer = setTimeout(() => onDismiss(toast.id), 3000);
    return () => clearTimeout(timer);
  }, [toast.id, onDismiss]);

  return (
    <div className={`toast toast-${toast.type}`}>
      <span>{toast.message}</span>
      <button className="toast-close" onClick={() => onDismiss(toast.id)}>
        ×
      </button>
    </div>
  );
}
```

frontend/src/App.jsx

```
import { useState, useEffect } from 'react'
import { Routes, Route } from 'react-router-dom'
```

```
import NavBar from './components/NavBar'
import Home from './pages/Home'
import Projects from './pages/Projects'
import Tasks from './pages/Tasks'
import Contact from './pages/Contact'
import NotFound from './pages/NotFound'
import './App.css'

function App() {
  const [darkMode, setDarkMode] = useState(true)

  useEffect(() => {
    document.documentElement.className = darkMode ? 'dark' : 'light'
  }, [darkMode])

  return (
    <div className="app-shell">
      <NavBar darkMode={darkMode} setDarkMode={setDarkMode} />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/projects" element={<Projects />} />
        <Route path="/tasks" element={<Tasks />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
```

```
  )
}

export default App
```

frontend/src/components/NavBar.jsx

```
<NavLink to="/tasks" className={({ isActive }) => isActive ? 'active' :
''}>Tasks</NavLink>
```

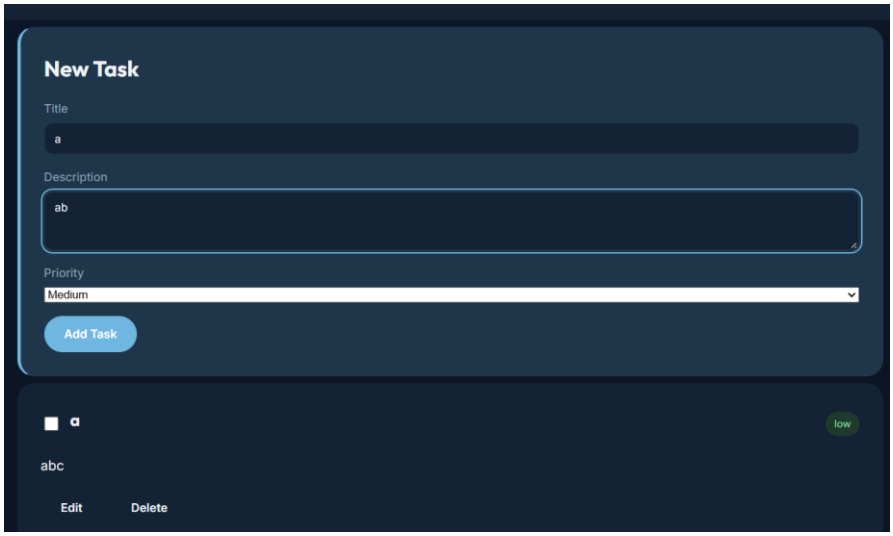
Supplementary Problems

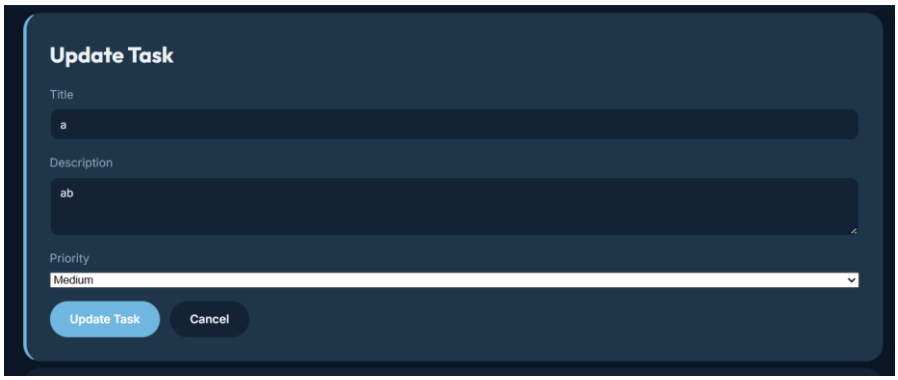
• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms.

Tasks.jsx:

```
const handleDelete = (task) => {
  const confirmed = window.confirm(`Delete "${task.title}"? This can't be
undone.`);
  if (!confirmed) return;

  setDeletingId(task._id);
  deleteTask(task._id)
    .then(() => {
      setTasks((prev) => prev.filter((t) => t._id !== task._id));
      addToast("Task deleted");
    })
    .catch((err) => addToast(`Delete failed: ${err.message}`, "error"))
    .finally(() => setDeletingId(null));
};
```



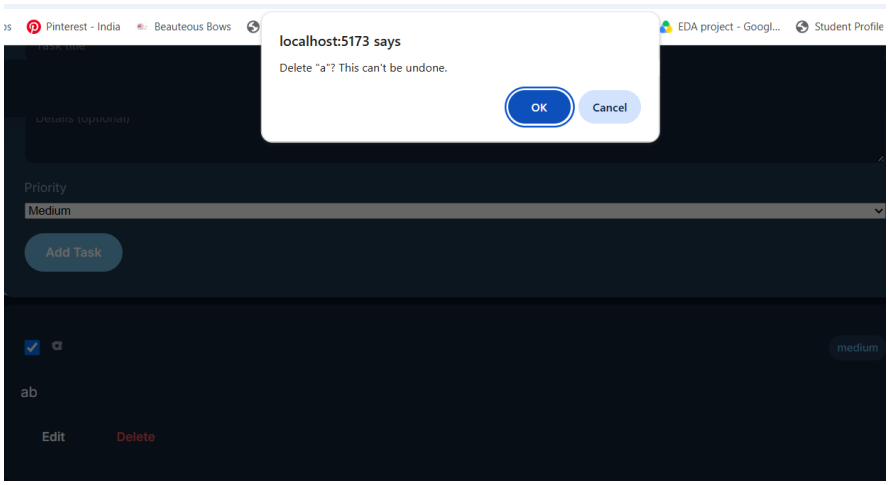


- **Add a confirmation dialog before deleting a task.**

```
// Show the task immediately with a temporary ID, before the server responds
const tempId = `temp-${Date.now()}`;
const optimisticTask = {
  _id: tempId,
  title: form.title,
  description: form.description,
  priority: form.priority,
  completed: false,
  isPending: true,
};
```

```
setTasks((prev) => [optimisticTask, ...prev]);
setForm(emptyForm);


createTask({ title: optimisticTask.title, description: optimisticTask.description,
priority: optimisticTask.priority })
  .then((savedTask) => {
    // Server confirmed — swap the temp card for the real saved task
    setTasks((prev) => prev.map((t) => (t._id === tempId ? savedTask : t)));
    addToast("Task added");
  })
  .catch((err) => {
    // Server rejected it — roll back the optimistic card
    setTasks((prev) => prev.filter((t) => t._id !== tempId));
    addToast(`Add failed: ${err.message}`, "error");
  });
```



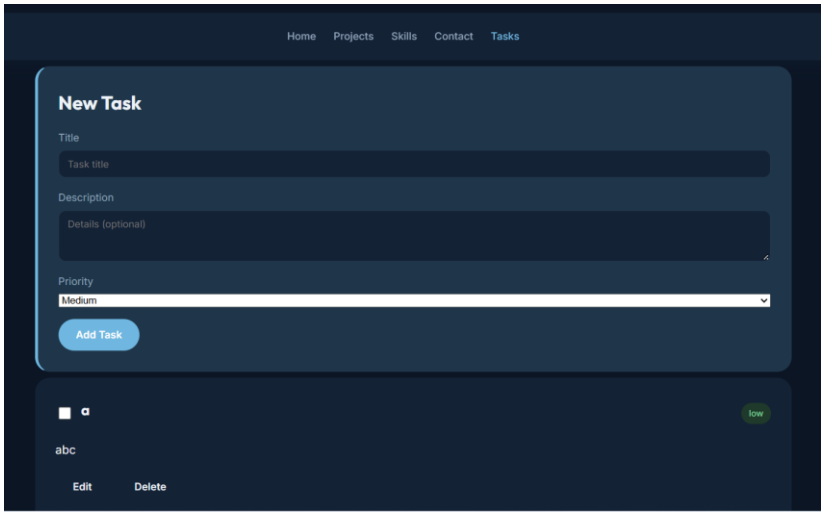
• Add a toast/notification component that shows success or failure after each operation.

```
import { useEffect } from "react";
import "./Toast.css";
```

```
export default function Toast({ toasts, onDismiss }) {
  return (
    <div className="toast-stack">
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} onDismiss={onDismiss} />
      ))}
    </div>
  );
}

function ToastItem({ toast, onDismiss }) {
  useEffect(() => {
    const timer = setTimeout(() => onDismiss(toast.id), 3000);
    return () => clearTimeout(timer);
  }, [toast.id, onDismiss]);

  return (
    <div className={`toast toast-${toast.type}`}>
      <span>{toast.message}</span>
      <button className="toast-close" onClick={() =>
onDismiss(toast.id)}>×</button>
    </div>
  );
}
```



Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF